

Simulacro III

Parcial II

Has sido contratado por BookVerse, una plataforma digital para gestionar libros y autores a nivel global. La empresa quiere que desarrolles un sistema frontend en Angular que permita a los usuarios administrar la información de la biblioteca a través de un API REST simulado con json-server.

Objetivos del examen:

Tu tarea es crear un proyecto Angular desde cero que utilice HttpClient para interactuar con el json-server y cuente con los componentes, servicios y enrutamientos necesarios. Para ello, se solicita:

1. Funcionalidades básicas:

- a) GET: Obtener la lista completa de libros y mostrarla en el componente.
- b) POST: Agregar un nuevo libro. Debes validar que todos los campos estén completos antes de enviarlo.
- c) PUT: Actualizar un libro existente mediante su id.
- d)
 - 1) En ambos formularios (POST y PUT), el campo género debe seleccionarse mediante un desplegable HTML <select>. Las opciones pueden incluir: Terror, Romance, Ciencia Ficción, Misterio, Fantástico.
Deberán investigar cómo usar la etiqueta <select> en Angular y cómo enlazarla al FormControl.
 - 2) En ambos formularios (POST y PUT), deberán usar 5 validadores, por ejemplo:
 - Validators.required → todos los campos obligatorios.
 - Validators.minLength(3) → el título debe tener al menos 3 caracteres.
 - Validators.maxLength(50) → el título no puede superar 50 caracteres.
 - Validators.pattern('[A-Za-z]+') → el autor solo puede contener letras y espacios.
 - Validators.min(1500) → el año de publicación debe ser mayor o igual a 1500.
- e) DELETE: Eliminar un libro existente mediante su id.

2) Funcionalidades extra:

- a) Mostrar un mensaje de confirmación al eliminar un libro y actualizar la lista automáticamente.
- b) Implementar un método que devuelva el libro más reciente según añoPublicacion.

c) Implementar un método `verificarLibroExistente` que permita no cargar libros duplicados en la colección. La verificación puede basarse en:

- id único del libro.
- O combinación de título + autor para evitar duplicados lógicos aunque el id cambie.

Este método debe aplicarse tanto al POST como al GET, para mantener la colección limpia y consistente.

3) Puntos a evaluar:

- Correcta implementación de las peticiones HTTP (GET, POST, PUT, DELETE) → 35 pts.
- Formulario de actualización y creación con 5 validadores y desplegable de categorías → 15 pts.
- Funcionalidades extra de obtención de libro más reciente → 10 pts.
- Correcta notificación al usuario mediante `alert()` o la forma deseada luego de cada acción o petición realizada. → 10 pts.
- Método `verificarLibroExistente` y manejo de duplicados → 10 pts.
- Manejo de errores y validaciones de datos → 10 pts.
- Presentación, modularidad del código y comentarios → 10 pts.